3.2 Software Architecture

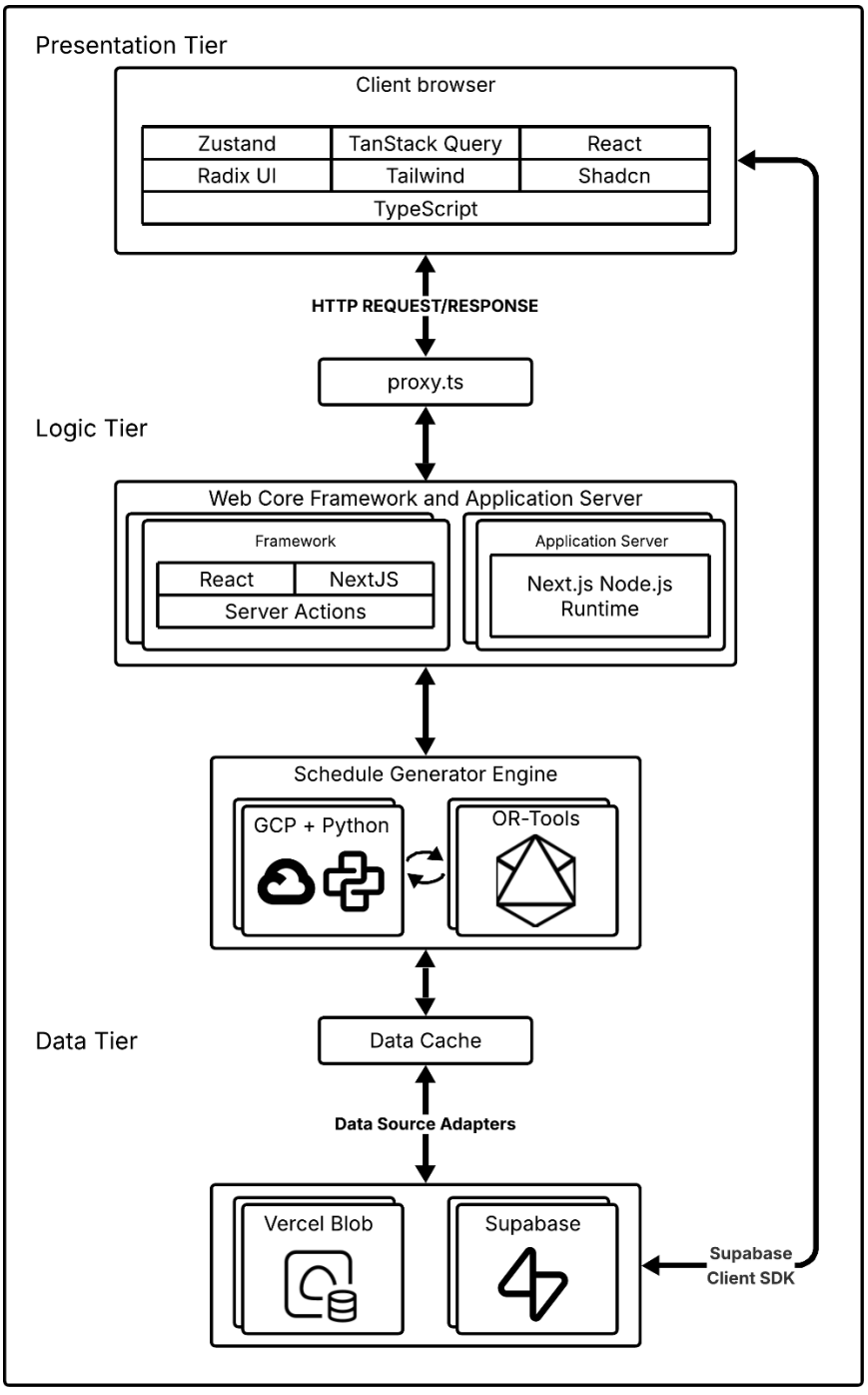


Figure 3.1.1: 3-Layered Software Architecture Diagram

This illustartes the 3-Layered Software Architecture of the proposed Web-Based Class Scheduler System. The architecture is composed of the Presentation Tier, Logic Tier, and Data Tier. The Presentation Tier serves as the user interface layer accessed through a client browser. It utilizes React 19, TypeScript, Shadcn, Radix UI, Tailwind CSS, Zustand, and TanStack Query to

provide an interactive, responsive, and efficient user experience. Next.js manages page rendering between the client and server, while proxy.ts acts as middleware that handles authentication validation and session management before route execution.

The Logic Tier contains the Web Core Framework and Application Server running on the Next.js Node.js runtime hosted on Vercel. Within this layer, Server Actions securely process user requests, perform database operations, and manage data revalidation. A key component of this tier is the Schedule Generator Engine, which automates the creation of class schedules. The engine utilizes Google Cloud Platform (GCP) with Python and Google OR-Tools to generate optimized schedules based on curriculum requirements, faculty workloads, section planning, room availability, and scheduling constraints. Through constraint-based optimization, the engine helps prevent conflicts involving faculty assignments, room utilization, and class schedules while improving the efficiency and accuracy of the scheduling process.

The Data Tier is responsible for data storage and management through Supabase and Vercel Blob. Supabase serves as the primary database and authentication provider, while Vercel Blob stores application files such as curriculum documents and uploaded COR files. A Data Cache layer is positioned between the Logic Tier and Data Tier to reduce redundant database requests and improve system performance. Data Source Adapters, implemented through the @supabase/ssr client, securely attach authenticated user credentials to all server-side requests, ensuring compliance with Row-Level Security (RLS) policies. Together, these components enable secure, scalable, and efficient data processing throughout the system.